Degrees of Uncomputability: Post's problem and the Finite Injury Priority method

Master's Degree in Computer Science

Simone Bianco (1986936)

Academic Year 2024/2025

SAPIENZA
UNIVERSITÀ DI ROMA

Roadmap

- Introduction on computation
- Degrees of Uncomputability
- The Finite Extension method
- The Finite Injury Priority method

Table of Contents

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems. It's the first mathematical definition of computation.

- *Constant functions*: return the natural number n
- *Successor function*: on input x return $x + 1$
- *Projection function*: on input $(x_1, \ldots, x_n)$ return x_i
- *Composition operator*: on input $h, g_1, \ldots, g_m$ return the function that applies h together on the outputs of the applications of $g_1, \ldots, g_m$
- *Primitive recursion operator*: on input h, g, return the function that applies g if the first input is 0 and otherwise applies h on the first argument x_0 and then recursively applies the process on the predecessor of x.
- *Minimization operator*: on input f, search the first value z that makes f return 0 when z is the first argument.

What is computation?

Defining computation

1931 – Kurt Gödel defines **General Recursive Functions** to prove his Incompleteness Theorems.

- *Constant functions*: $C_n^k(x_1, \ldots, x_k) := n$ for all $k, n \in \mathbb{N}$
- *Successor function*: $S(x) := x + 1$
- *Projection function*: $P_i^k(x_1, \ldots, x_k) := x_i$ for all $k \in \mathbb{N}$
- *Composition operator*:
  $(h \circ (g_1, \ldots, g_m))(x_1, \ldots, x_k) := h(g_1(x_1, \ldots, x_k), \ldots, g_m(x_1, \ldots, x_k))$
- *Primitive recursion operator*: $\rho(g, h) := f$ where:

$$f(x_0, x_1, \ldots, x_k) = \begin{cases} g(x_1, \ldots, x_k) & \text{if } x_0 = 0 \\ h(y_0, f(y_0, x_1 \ldots, x_k)) & \text{if } x_0 = S(y_0) \end{cases}$$

- *Minimization operator*: $\mu(f) := f'$ where:

$$f'(x_1, \ldots, x_k) = \begin{cases} z & \text{if } f(z, x_1, \ldots, x_k) > 0 \text{ and } f(i, x_1, \ldots, x_k) > 0 \text{ for all } i < z \\ * & \text{otherwise} \end{cases}$$

What is computation?

Defining computation

1935 – Alonzo Church defines λ**-calculus** to settle the Entscheidungsproblem problem.

- *Language grammar*: $M, N ::= x \mid \lambda x.M \mid MN$
- *Substitution*: $M[x := N]$ means that x gets substituted by N in M
- α*-rule*: $\lambda x.M \xrightarrow{\alpha} \lambda y.(M[x := y])$
- β*-rule*: $(\lambda x.M)N \xrightarrow{\beta} M[x := N]$
- η*-rule*: $\lambda x.Mx \xrightarrow{\eta} M$ if $x \notin \text{free}(M)$

Stephen Kleene proves that λ-calculus is equivalent to general recursive functions

What is computation?

Defining computation

1935 – Alonzo Church defines λ**-calculus** to settle the Entscheidungsproblem problem.

- *Language grammar*: $M, N ::= x \mid \lambda x.M \mid MN$
- *Substitution*: $M[x := N]$ means that x gets substituted by N in M
- α*-rule*: $\lambda x.M \xrightarrow{\alpha} \lambda y.(M[x := y])$
- β*-rule*: $(\lambda x.M)N \xrightarrow{\beta} M[x := N]$
- η*-rule*: $\lambda x.Mx \xrightarrow{\eta} M$ if $x \notin \text{free}(M)$

Stephen Kleene proves that λ-calculus is equivalent to general recursive functions

What is computation?

Defining computation

1935 – Alonzo Church defines λ**-calculus** to settle the Entscheidungsproblem problem.

- *Language grammar*: $M, N ::= x \mid \lambda x.M \mid MN$
- *Substitution*: $M[x := N]$ means that x gets substituted by N in M
- α*-rule*: $\lambda x.M \xrightarrow{\alpha} \lambda y.(M[x := y])$
- β*-rule*: $(\lambda x.M)N \xrightarrow{\beta} M[x := N]$
- η*-rule*: $\lambda x.Mx \xrightarrow{\eta} M$ if $x \notin \text{free}(M)$

Stephen Kleene proves that λ-calculus is equivalent to general recursive functions

What is computation?

Defining computation

1935 – Alonzo Church defines λ**-calculus** to settle the Entscheidungsproblem problem.

- *Language grammar*: $M, N ::= x \mid \lambda x.M \mid MN$
- *Substitution*: $M[x := N]$ means that x gets substituted by N in M
- α*-rule*: $\lambda x.M \xrightarrow{\alpha} \lambda y.(M[x := y])$
- β*-rule*: $(\lambda x.M)N \xrightarrow{\beta} M[x := N]$
- η*-rule*: $\lambda x.Mx \xrightarrow{\eta} M$ if $x \notin \text{free}(M)$

Stephen Kleene proves that λ-calculus is equivalent to general recursive functions

What is computation?

Defining computation

1935 – Alonzo Church defines λ**-calculus** to settle the Entscheidungsproblem problem.

- *Language grammar*: $M, N ::= x \mid \lambda x.M \mid MN$
- *Substitution*: $M[x := N]$ means that x gets substituted by N in M
- α*-rule*: $\lambda x.M \xrightarrow{\alpha} \lambda y.(M[x := y])$
- β*-rule*: $(\lambda x.M)N \xrightarrow{\beta} M[x := N]$
- η*-rule*: $\lambda x.Mx \xrightarrow{\eta} M$ if $x \notin \text{free}(M)$

Stephen Kleene proves that λ-calculus is equivalent to general recursive functions

What is computation?

Defining computation

1935 – Alonzo Church defines λ**-calculus** to settle the Entscheidungsproblem problem.

- *Language grammar*: $M, N ::= x \mid \lambda x.M \mid MN$
- *Substitution*: $M[x := N]$ means that x gets substituted by N in M
- α*-rule*: $\lambda x.M \xrightarrow{\alpha} \lambda y.(M[x := y])$
- β*-rule*: $(\lambda x.M)N \xrightarrow{\beta} M[x := N]$
- η*-rule*: $\lambda x.Mx \xrightarrow{\eta} M$ if $x \notin \text{free}(M)$

Stephen Kleene proves that λ-calculus is equivalent to general recursive functions

What is computation?

Defining computation

1936 – Alan Turing defines a simple abstract model of a computer.

- Infinite tape subdivided in cells with a read-write head
- Set of instructions defined by a function δ
- Each instruction moves the head left or right based on the current state and the value read.
- It may also change the value below the head before moving.

Turing's machine also settles the Entscheidungsproblem and it is equivalent to λ-calculus. Recognized by Gödel himself as the best definition of computation.

What is computation?

Defining computation

1936 – Alan Turing defines a simple abstract model of a computer.

- Infinite tape subdivided in cells with a read-write head
- Set of instructions defined by a function δ
- Each instruction moves the head left or right based on the current state and the value read.
- It may also change the value below the head before moving.

Turing's machine also settles the Entscheidungsproblem and it is equivalent to λ-calculus. Recognized by Gödel himself as the best definition of computation.

What is computation?

Defining computation

1936 – Alan Turing defines a simple abstract model of a computer.

- Infinite tape subdivided in cells with a read-write head
- Set of instructions defined by a function δ
- Each instruction moves the head left or right based on the current state and the value read.
- It may also change the value below the head before moving.

Turing's machine also settles the Entscheidungsproblem and it is equivalent to λ-calculus. Recognized by Gödel himself as the best definition of computation.

What is computation?

Defining computation

1936 – Alan Turing defines a simple abstract model of a computer.

- Infinite tape subdivided in cells with a read-write head
- Set of instructions defined by a function δ
- Each instruction moves the head left or right based on the current state and the value read.
- It may also change the value below the head before moving.

Turing's machine also settles the Entscheidungsproblem and it is equivalent to λ-calculus. Recognized by Gödel himself as the best definition of computation.

What is computation?

Defining computation

1936 – Alan Turing defines a simple abstract model of a computer.

- Infinite tape subdivided in cells with a read-write head
- Set of instructions defined by a function δ
- Each instruction moves the head left or right based on the current state and the value read.
- It may also change the value below the head before moving.

Turing's machine also settles the Entscheidungsproblem and it is equivalent to λ-calculus. Recognized by Gödel himself as the best definition of computation.

What is computation?

Defining computation

$\Phi_{i,s}(x)$ denotes the **computation** of the i-th TM for $s \in \mathbb{N}$ steps on input $x \in \mathbb{N}$.

- $\Phi_{i,s}(x) \downarrow$: the computation halts after s steps
- $\Phi_i(x) \downarrow$: there is a $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\Phi_i(x) \uparrow$: there is no $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\psi_i(x)$: output of the computation (defined iff $\Phi_i(x) \downarrow$)

What is computation?

Defining computation

$\Phi_{i,s}(x)$ denotes the **computation** of the i-th TM for $s \in \mathbb{N}$ steps on input $x \in \mathbb{N}$.

- $\Phi_{i,s}(x) \downarrow$: the computation halts after s steps
- $\Phi_i(x) \downarrow$: there is a $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\Phi_i(x) \uparrow$: there is no $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\psi_i(x)$: output of the computation (defined iff $\Phi_i(x) \downarrow$)

What is computation?

Defining computation

$\Phi_{i,s}(x)$ denotes the **computation** of the i-th TM for $s \in \mathbb{N}$ steps on input $x \in \mathbb{N}$.

- $\Phi_{i,s}(x) \downarrow$: the computation halts after s steps
- $\Phi_i(x) \downarrow$: there is a $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\Phi_i(x) \uparrow$: there is no $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\psi_i(x)$: output of the computation (defined iff $\Phi_i(x) \downarrow$)

What is computation?

Defining computation

$\Phi_{i,s}(x)$ denotes the **computation** of the i-th TM for $s \in \mathbb{N}$ steps on input $x \in \mathbb{N}$.

- $\Phi_{i,s}(x) \downarrow$: the computation halts after s steps
- $\Phi_i(x) \downarrow$: there is a $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\Phi_i(x) \uparrow$: there is no $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\psi_i(x)$: output of the computation (defined iff $\Phi_i(x) \downarrow$)

What is computation?

Defining computation

$\Phi_{i,s}(x)$ denotes the **computation** of the i-th TM for $s \in \mathbb{N}$ steps on input $x \in \mathbb{N}$.

- $\Phi_{i,s}(x) \downarrow$: the computation halts after s steps
- $\Phi_i(x) \downarrow$: there is a $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\Phi_i(x) \uparrow$: there is no $s \in \mathbb{N}$ such that $\Phi_{i,s}(x) \downarrow$
- $\psi_i(x)$: output of the computation (defined iff $\Phi_i(x) \downarrow$)

Computability

Definitions

Given a set $S \subseteq \mathbb{N}$, we say that S is:

- **Semi-decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$ or $\Phi_i(x) \uparrow$.
- **Decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$.
- **Recursively enumerable** when there is an algorithmic procedure $A : \mathbb{N} \to \{0, 1\}$ such that $S = \{A(0), A(1), A(2), \ldots\}$

Obs. 1: S is semi-decidable if and only if it is recursively enumerable
Obs. 2: S is decidable if and only if both S and $\overline{S}$ are semi-decidable.

Computability

Definitions

Given a set $S \subseteq \mathbb{N}$, we say that S is:

- **Semi-decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$ or $\Phi_i(x) \uparrow$.
- **Decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$.
- **Recursively enumerable** when there is an algorithmic procedure $\mathcal{A} : \mathbb{N} \to \{0, 1\}$ such that $S = \{A(0), A(1), A(2), \ldots\}$

Obs. 1: S is semi-decidable if and only if it is recursively enumerable
Obs. 2: S is decidable if and only if both S and $\overline{S}$ are semi-decidable.

Computability

Definitions

Given a set $S \subseteq \mathbb{N}$, we say that S is:

- **Semi-decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$ or $\Phi_i(x) \uparrow$.
- **Decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$.
- **Recursively enumerable** when there is an algorithmic procedure $\mathcal{A} : \mathbb{N} \to \{0, 1\}$ such that $S = \{A(0), A(1), A(2), \ldots\}$

Obs. 1: S is semi-decidable if and only if it is recursively enumerable
Obs. 2: S is decidable if and only if both S and $\overline{S}$ are semi-decidable.

Computability

Definitions

Given a set $S \subseteq \mathbb{N}$, we say that S is:

- **Semi-decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$ or $\Phi_i(x) \uparrow$.
- **Decidable** when $\exists i \in \mathbb{N}$ such that $\forall x \in \mathbb{N}$ it holds that $\psi_i(x) = 1$ if $x \in S$, otherwise $\psi_i(x) = 0$.
- **Recursively enumerable** when there is an algorithmic procedure $\mathcal{A} : \mathbb{N} \to \{0, 1\}$ such that $S = \{A(0), A(1), A(2), \ldots\}$

Obs. 1: S is semi-decidable if and only if it is recursively enumerable
Obs. 2: S is decidable if and only if both S and $\overline{S}$ are semi-decidable.

Introduction

Turing's work

Turing [Tur36] proved that:

- Some sets that are semi-decidable but undecidable (e.g. $H = \{\langle i, x\rangle \mid \Phi_i(x) \downarrow\}$).
- Some sets cannot be semi-decided (e.g. $\overline{H}$).

Three degrees of computability: **solvable** problems, **semi-solvable** problems and **unsolvable** problems.

Are there some other degrees of computability?

Introduction

Turing's work

Turing [Tur36] proved that:

- Some sets that are semi-decidable but undecidable (e.g. $H = \{\langle i, x\rangle \mid \Phi_i(x) \downarrow\}$).
- Some sets cannot be semi-decided (e.g. $\overline{H}$).

Three degrees of computability: **solvable** problems, **semi-solvable** problems and **unsolvable** problems.

Are there some other degrees of computability?

Introduction

Turing's work

Turing [Tur36] proved that:

- Some sets that are semi-decidable but undecidable (e.g. $H = \{\langle i, x\rangle \mid \Phi_i(x) \downarrow\}$).
- Some sets cannot be semi-decided (e.g. $\overline{H}$).

Three degrees of computability: **solvable** problems, **semi-solvable** problems and **unsolvable** problems.

Are there some other degrees of computability?

Table of Contents

2 Degrees of uncomputability

Degrees of uncomputability

Turing degrees

Post [Pos44] formalized the idea of computability degrees through **Turing reductions**.

Let Φ_i^B denote the Turing machine Φ_i with access to an oracle machine that decides B.

- *Turing reducibility*: $A \leq_T B$ when $\exists i \in \mathbb{N}$ such that $\Phi_i^B(x) \downarrow$ for all $x \in \mathbb{N}$ and $A = \{x \in \mathbb{N} \mid \psi_i^B(x) = 1\}$.
- *Turing equivalence*: $A \equiv_T B$ when $A \leq_T B$ and $B \leq_T A$
- The set $\mathcal{D} = 2^{\mathbb{N}}/_{\equiv_T}$ is the set of **Turing degrees**.
- $[A]$ is the class containing A over $\mathcal{D}$.

Degrees of uncomputability

Turing degrees

Post [Pos44] formalized the idea of computability degrees through **Turing reductions**.

Let Φ_i^B denote the Turing machine Φ_i with access to an oracle machine that decides B.

- *Turing reducibility*: $A \leq_T B$ when $\exists i \in \mathbb{N}$ such that $\Phi_i^B(x) \downarrow$ for all $x \in \mathbb{N}$ and $A = \{x \in \mathbb{N} \mid \psi_i^B(x) = 1\}$.
- *Turing equivalence*: $A \equiv_T B$ when $A \leq_T B$ and $B \leq_T A$
- The set $\mathcal{D} = 2^{\mathbb{N}}/_{\equiv_T}$ is the set of **Turing degrees**.
- $[A]$ is the class containing A over $\mathcal{D}$.

Degrees of uncomputability

Turing degrees

Post [Pos44] formalized the idea of computability degrees through **Turing reductions**.

Let Φ_i^B denote the Turing machine Φ_i with access to an oracle machine that decides B.

- *Turing reducibility*: $A \leq_T B$ when $\exists i \in \mathbb{N}$ such that $\Phi_i^B(x) \downarrow$ for all $x \in \mathbb{N}$ and $A = \{x \in \mathbb{N} \mid \psi_i^B(x) = 1\}$.
- *Turing equivalence*: $A \equiv_T B$ when $A \leq_T B$ and $B \leq_T A$
- The set $\mathcal{D} = 2^{\mathbb{N}}/_{\equiv_T}$ is the set of **Turing degrees**.
- $[A]$ is the class containing A over $\mathcal{D}$.

Degrees of uncomputability

Turing degrees

Post [Pos44] formalized the idea of computability degrees through **Turing reductions**.

Let Φ_i^B denote the Turing machine Φ_i with access to an oracle machine that decides B.

- *Turing reducibility*: $A \leq_T B$ when $\exists i \in \mathbb{N}$ such that $\Phi_i^B(x) \downarrow$ for all $x \in \mathbb{N}$ and $A = \{x \in \mathbb{N} \mid \psi_i^B(x) = 1\}$.
- *Turing equivalence*: $A \equiv_T B$ when $A \leq_T B$ and $B \leq_T A$
- The set $\mathcal{D} = 2^{\mathbb{N}}/_{\equiv_T}$ is the set of **Turing degrees**.
- $[A]$ is the class containing A over $\mathcal{D}$.

Degrees of uncomputability

Turing degrees

Post [Pos44] formalized the idea of computability degrees through **Turing reductions**.

Let Φ_i^B denote the Turing machine Φ_i with access to an oracle machine that decides B.

- *Turing reducibility*: $A \leq_T B$ when $\exists i \in \mathbb{N}$ such that $\Phi_i^B(x) \downarrow$ for all $x \in \mathbb{N}$ and $A = \{x \in \mathbb{N} \mid \psi_i^B(x) = 1\}$.
- *Turing equivalence*: $A \equiv_T B$ when $A \leq_T B$ and $B \leq_T A$
- The set $\mathcal{D} = 2^{\mathbb{N}}/_{\equiv_T}$ is the set of **Turing degrees**.
- $[A]$ is the class containing A over $\mathcal{D}$.

Degrees of uncomputability

Turing degrees

We say that $[A]$ is **lower** than $[B]$, written as $[A] \preceq [B]$, if $A \leq_T B$.

$\preceq$ forms an ordering over the set $\mathcal{D}$, giving an hierarchy of unsolvability degrees.

$0 = [\varnothing]$ is the unique degree containing all decidable problems

- There is no degree below 0
- 0 is lower than any degree

Degrees of uncomputability

Turing degrees

We say that $[A]$ is **lower** than $[B]$, written as $[A] \preceq [B]$, if $A \leq_T B$.

$\preceq$ forms an ordering over the set $\mathcal{D}$, giving an hierarchy of unsolvability degrees.

$0 = [\varnothing]$ is the unique degree containing all decidable problems

- There is no degree below 0
- 0 is lower than any degree

Degrees of uncomputability

Turing degrees

We say that $[A]$ is **lower** than $[B]$, written as $[A] \preceq [B]$, if $A \leq_T B$.

$\preceq$ forms an ordering over the set $\mathcal{D}$, giving an hierarchy of unsolvability degrees.

$0 = [\varnothing]$ is the unique degree containing all decidable problems

- There is no degree below 0
- 0 is lower than any degree

Degrees of uncomputability

Turing degrees

We say that $[A]$ is **lower** than $[B]$, written as $[A] \preceq [B]$, if $A \leq_T B$.

$\preceq$ forms an ordering over the set $\mathcal{D}$, giving an hierarchy of unsolvability degrees.

$0 = [\varnothing]$ is the unique degree containing all decidable problems

- There is no degree below 0
- 0 is lower than any degree

Degrees of uncomputability

Turing degrees

We say that $[A]$ is **lower** than $[B]$, written as $[A] \preceq [B]$, if $A \leq_T B$.

$\preceq$ forms an ordering over the set $\mathcal{D}$, giving an hierarchy of unsolvability degrees.

$0 = [\varnothing]$ is the unique degree containing all decidable problems

- There is no degree below 0
- 0 is lower than any degree

Degrees of uncomputability

The jump operator

The strict relation $[A] \prec [B]$ between degrees can be easily forced through **Turing jumps**.

- Given a set $X \subseteq \mathbb{N}$, the *Turing jump* of X is the set $X' = \{\langle i, x\rangle \mid \Phi_i^X(x) \downarrow\}$
  - $X \leq_T X'$ but $X' \not\leq_T X$ since X' is a variant of the Halting problem using X as an oracle.
- The jump of $\varnothing$ is exactly the Halting problem H
- We define $0'$ as the class of H (or $\varnothing'$)

Degrees of uncomputability

The jump operator

The strict relation $[A] \prec [B]$ between degrees can be easily forced through **Turing jumps**.

- Given a set $X \subseteq \mathbb{N}$, the *Turing jump* of X is the set $X' = \{\langle i, x\rangle \mid \Phi_i^X(x) \downarrow\}$
  - $X \leq_T X'$ but $X' \not\leq_T X$ since X' is a variant of the Halting problem using X as an oracle.

- The jump of $\varnothing$ is exactly the Halting problem H
- We define $0'$ as the class of H (or $\varnothing'$)

Degrees of uncomputability

The jump operator

The strict relation $[A] \prec [B]$ between degrees can be easily forced through **Turing jumps**.

- Given a set $X \subseteq \mathbb{N}$, the *Turing jump* of X is the set $X' = \{\langle i, x\rangle \mid \Phi_i^X(x) \downarrow\}$
  - $X \leq_T X'$ but $X' \not\leq_T X$ since X' is a variant of the Halting problem using X as an oracle.

- The jump of $\varnothing$ is exactly the Halting problem H
- We define $0'$ as the class of H (or $\varnothing'$)

Degrees of uncomputability

The jump operator

The strict relation $[A] \prec [B]$ between degrees can be easily forced through **Turing jumps**.

- Given a set $X \subseteq \mathbb{N}$, the *Turing jump* of X is the set $X' = \{\langle i, x\rangle \mid \Phi_i^X(x) \downarrow\}$
  - $X \leq_T X'$ but $X' \not\leq_T X$ since X' is a variant of the Halting problem using X as an oracle.

- The jump of $\varnothing$ is exactly the Halting problem H
- We define $0'$ as the class of H (or $\varnothing'$)

Degrees of uncomputability

Low degrees

- The jump operator can be iteratively applied to get infinite levels of the hierarchy
- All r.e. degrees are below $0'$
  - A degree is said to be r.e. if it contains an r.e. set
  - Not all sets inside an r.e. degree are r.e. (e.g. $\overline{H} \in 0'$)
- Degrees below $0'$ are referred to as **low degrees**
  - Being below $0'$ doesn't guarantee being r.e.!

Degrees of uncomputability

Low degrees

- The jump operator can be iteratively applied to get infinite levels of the hierarchy
- All r.e. degrees are below $0'$
  - A degree is said to be r.e. if it contains an r.e. set
  - Not all sets inside an r.e. degree are r.e. (e.g. $\overline{H} \in 0'$)
- Degrees below $0'$ are referred to as **low degrees**
  - Being below $0'$ doesn't guarantee being r.e.!

Degrees of uncomputability

Low degrees

- The jump operator can be iteratively applied to get infinite levels of the hierarchy
- All r.e. degrees are below $0'$
  - A degree is said to be r.e. if it contains an r.e. set
  - Not all sets inside an r.e. degree are r.e. (e.g. $\overline{H} \in 0'$)
- Degrees below $0'$ are referred to as **low degrees**
  - Being below $0'$ doesn't guarantee being r.e.!

Degrees of uncomputability

Low degrees

- The jump operator can be iteratively applied to get infinite levels of the hierarchy
- All r.e. degrees are below $0'$
  - A degree is said to be r.e. if it contains an r.e. set
  - Not all sets inside an r.e. degree are r.e. (e.g. $\overline{H} \in 0'$)
- Degrees below $0'$ are referred to as **low degrees**
  - Being below $0'$ doesn't guarantee being r.e.!

Degrees of uncomputability

Low degrees

- The jump operator can be iteratively applied to get infinite levels of the hierarchy
- All r.e. degrees are below $0'$
  - A degree is said to be r.e. if it contains an r.e. set
  - Not all sets inside an r.e. degree are r.e. (e.g. $\overline{H} \in 0'$)
- Degrees below $0'$ are referred to as **low degrees**
  - Being below $0'$ doesn't guarantee being r.e.!

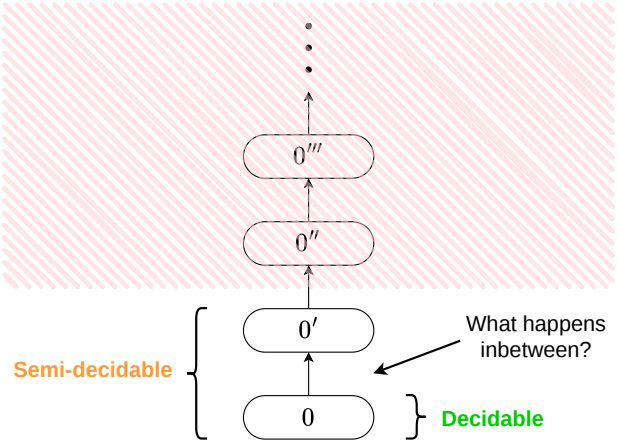
Degrees of uncomputability

Low degrees

- The jump operator can be iteratively applied to get infinite levels of the hierarchy
- All r.e. degrees are below $0'$
  - A degree is said to be r.e. if it contains an r.e. set
  - Not all sets inside an r.e. degree are r.e. (e.g. $\overline{H} \in 0'$)
- Degrees below $0'$ are referred to as **low degrees**
  - Being below $0'$ doesn't guarantee being r.e.!

Degrees of uncomputability

Low degrees



Degrees of uncomputability

Post's problem

- In [Pos44], Emil Post investigates the existence of a degree d such that $0 \prec d \prec 0'$
- He was able to prove that the existence of such degree,
- However, he couldn't find a way to force the degree to be r.e.

Post's problem: is there a r.e. degree d such that $0 \prec d \prec 0'$?

Degrees of uncomputability

Post's problem

- In [Pos44], Emil Post investigates the existence of a degree d such that $0 \prec d \prec 0'$
- He was able to prove that the existence of such degree,
- However, he couldn't find a way to force the degree to be r.e.

Post's problem: is there a r.e. degree d such that $0 \prec d \prec 0'$?

Degrees of uncomputability

Post's problem

- In [Pos44], Emil Post investigates the existence of a degree d such that $0 \prec d \prec 0'$
- He was able to prove that the existence of such degree,
- However, he couldn't find a way to force the degree to be r.e.

Post's problem: is there a r.e. degree d such that $0 \prec d \prec 0'$?

Degrees of uncomputability

Post's problem

- In [Pos44], Emil Post investigates the existence of a degree d such that $0 \prec d \prec 0'$
- He was able to prove that the existence of such degree,
- However, he couldn't find a way to force the degree to be r.e.

Post's problem: is there a r.e. degree d such that $0 \prec d \prec 0'$?

Table of Contents

The Finite Extension method

Towards the solution to Post's problem

- Post's problem was solved 12 years later independently by Friedberg [Fri57] and Muchnik [Muc56] through the **finite injury priority method**.
- The method is an improvement on the **finite extension method** developed by Post in his original works
- Since the FIP method involves constructions that are way more complex than those of the FE method, we'll first give an example of the latter

The Finite Extension method

Towards the solution to Post's problem

- Post's problem was solved 12 years later independently by Friedberg [Fri57] and Muchnik [Muc56] through the **finite injury priority method**.
- The method is an improvement on the **finite extension method** developed by Post in his original works
- Since the FIP method involves constructions that are way more complex than those of the FE method, we'll first give an example of the latter

The Finite Extension method

Towards the solution to Post's problem

- Post's problem was solved 12 years later independently by Friedberg [Fri57] and Muchnik [Muc56] through the **finite injury priority method**.
- The method is an improvement on the **finite extension method** developed by Post in his original works
- Since the FIP method involves constructions that are way more complex than those of the FE method, we'll first give an example of the latter

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings $\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers "selected" by s

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings$\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers "selected" by s

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings$\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers "selected" by s

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings$\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers "selected" by s

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings$\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers "selected" by s

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings$\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers "selected" by s

The Finite Extension method

The idea

- We define a list of requirements $\{R_i\}_{i\in\mathbb{N}}$ that will enforce our desired property over the sets A and B
- We construct two list of strings$\{\alpha_i\}_{i\in\mathbb{N}}$ and $\{\beta_i\}_{i\in\mathbb{N}}$ where:
  - α_i is a finite extension of α_{i-1} (same goes for β_i and β_{i-1})
  - The strings α_i, β_i will satisfy requirement R_i
  - Strings are to be considered as partial functions from $\mathbb{N}$ to $\{0, 1\}$ (equivalently, as an infinite tape over $\{0, 1, *\}$)
- The sets $A = \bigcup_{i=0}^{+\infty} \mathbb{N}[\alpha_i]$ and $B = \bigcup_{i=0}^{+\infty} \mathbb{N}[\beta_i]$ will have our desired property
  - $\mathbb{N}[s] = \{x \in \mathbb{N} \mid s(x) = 1\}$ is the set of all natural numbers “selected” by s

The Finite Extension method

The Kleene-Post theorem

Theorem

[KP54] There are two low sets A and B that are incomparable

- Requirements:

$$R_{2j} := (\psi_j^B \neq A) \qquad R_{2j+1} := (\psi_j^A \neq B)$$

- Start by setting $\alpha_0 = \beta_0 = \varepsilon$

The Finite Extension method

The Kleene-Post theorem

Theorem

[KP54] There are two low sets A and B that are incomparable

- Requirements:

$$R_{2j} := (\psi_j^B \neq A) \qquad R_{2j+1} := (\psi_j^A \neq B)$$

- Start by setting $\alpha_0 = \beta_0 = \varepsilon$

The Finite Extension method

The Kleene-Post theorem

- Consider any index $2j > 0$. Pick any position $x \in \mathbb{N}$ such that $\beta_{2j-1}(x) = *$ (guaranteed to exist)
- If there is any finite extension α' of α_{2j-1} such that $\Phi_j^{\{\alpha'\}}(x) \downarrow$, we set:

$$\alpha_{2j} = \alpha' \qquad \beta_{2j}(y) = \begin{cases} 1 - \psi_j^{\{\alpha'\}}(x) & \text{if } y = x \\ \beta_{2j-1}(y) & \text{otherwise} \end{cases}$$

- Otherwise, we set:

$$\alpha_{2j} = \alpha_{2j-1} \qquad \beta_{2j}(y) = \begin{cases} \text{rand}(\{0,1\}) & \text{if } y = x \\ \beta_{2j-1}(y) & \text{otherwise} \end{cases}$$

The Finite Extension method

The Kleene-Post theorem

- Consider any index $2j > 0$. Pick any position $x \in \mathbb{N}$ such that $\beta_{2j-1}(x) = *$ (guaranteed to exist)
- If there is any finite extension α' of α_{2j-1} such that $\Phi_j^{\{\alpha'\}}(x)\downarrow$, we set:

$$\alpha_{2j} = \alpha' \qquad \beta_{2j}(y) = \begin{cases} 1 - \psi_j^{\{\alpha'\}}(x) & \text{if } y = x \\ \beta_{2j-1}(y) & \text{otherwise} \end{cases}$$

- Otherwise, we set:

$$\alpha_{2j} = \alpha_{2j-1} \qquad \beta_{2j}(y) = \begin{cases} \mathrm{rand}(\{0,1\}) & \text{if } y = x \\ \beta_{2j-1}(y) & \text{otherwise} \end{cases}$$

The Finite Extension method

The Kleene-Post theorem

- Consider any index $2j > 0$. Pick any position $x \in \mathbb{N}$ such that $\beta_{2j-1}(x) = *$ (guaranteed to exist)
- If there is any finite extension α' of α_{2j-1} such that $\Phi_j^{\{\alpha'\}}(x)\downarrow$, we set:

$$\alpha_{2j} = \alpha' \qquad \beta_{2j}(y) = \begin{cases} 1 - \psi_j^{\{\alpha'\}}(x) & \text{if } y = x \\ \beta_{2j-1}(y) & \text{otherwise} \end{cases}$$

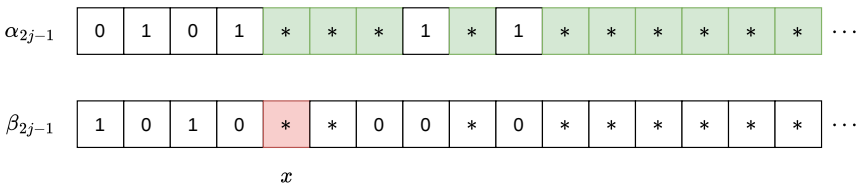
- Otherwise, we set:

$$\alpha_{2j} = \alpha_{2j-1} \qquad \beta_{2j}(y) = \begin{cases} \mathrm{rand}(\{0,1\}) & \text{if } y = x \\ \beta_{2j-1}(y) & \text{otherwise} \end{cases}$$

The Finite Extension method

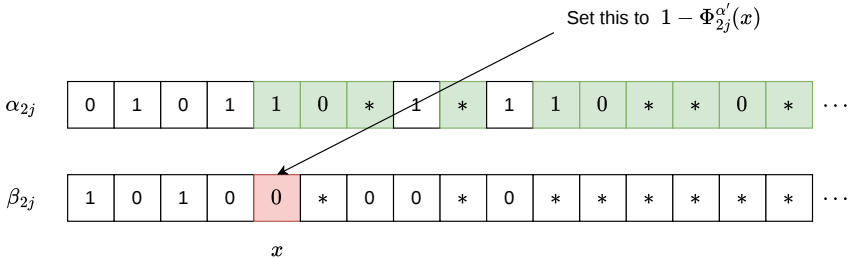
The Kleene-Post theorem

Is there a way to set a finite number of these bits in a way such that $\Phi_{2j}^{\alpha'}(x)$ terminates?



The Finite Extension method

The Kleene-Post theorem



The Finite Extension method

The Kleene-Post theorem

- Strings α_{2j}, β_{2j} satisfy requirement R_{2j}
- Symmetrical idea applied to index $2j+1 > 0$, satisfying R_{2j+1}
- **Incomparability**: all requirements are satisfied
- **Lowness**: A and B can be enumerated through an oracle for H but not without it

□

The Finite Extension method

The Kleene-Post theorem

- Strings α_{2j}, β_{2j} satisfy requirement R_{2j}
- Symmetrical idea applied to index $2j + 1 > 0$, satisfying R_{2j+1}
- **Incomparability**: all requirements are satisfied
- **Lowness**: A and B can be enumerated through an oracle for H but not without it

□

The Finite Extension method

The Kleene-Post theorem

- Strings α_{2j}, β_{2j} satisfy requirement R_{2j}
- Symmetrical idea applied to index $2j + 1 > 0$, satisfying R_{2j+1}
- **Incomparability**: all requirements are satisfied
- **Lowness**: A and B can be enumerated through an oracle for H but not without it

□

The Finite Extension method

The Kleene-Post theorem

- Strings α_{2j}, β_{2j} satisfy requirement R_{2j}
- Symmetrical idea applied to index $2j + 1 > 0$, satisfying R_{2j+1}
- **Incomparability**: all requirements are satisfied
- **Lowness**: A and B can be enumerated through an oracle for H but not without it

□

Table of Contents

The Finite Injury Priority method

The idea

- **Injury**: a requirement gets injured when its satisfaction is not guaranteed anymore.
- The key idea is to assign a priority level to each requirement:
  1. Each requirement will have finitely many requirements with higher priority
  2. Each requirement will be injured only by requirements with higher priority
- We'll use *sets* instead of strings to construct A and B

The Finite Injury Priority method

The idea

- **Injury**: a requirement gets injured when its satisfaction is not guaranteed anymore.
- The key idea is to assign a priority level to each requirement:
  1. Each requirement will have finitely many requirements with higher priority
  2. Each requirement will be injured only by requirements with higher priority
- We'll use *sets* instead of strings to construct A and B

The Finite Injury Priority method

The idea

- **Injury**: a requirement gets injured when its satisfaction is not guaranteed anymore.
- The key idea is to assign a priority level to each requirement:
  1. Each requirement will have finitely many requirements with higher priority
  2. Each requirement will be injured only by requirements with higher priority
- We'll use *sets* instead of strings to construct A and B

The Finite Injury Priority method

The idea

- **Injury**: a requirement gets injured when its satisfaction is not guaranteed anymore.
- The key idea is to assign a priority level to each requirement:
  1. Each requirement will have finitely many requirements with higher priority
  2. Each requirement will be injured only by requirements with higher priority
- We'll use *sets* instead of strings to construct A and B

The Finite Injury Priority method

The idea

- **Injury**: a requirement gets injured when its satisfaction is not guaranteed anymore.
- The key idea is to assign a priority level to each requirement:
  1. Each requirement will have finitely many requirements with higher priority
  2. Each requirement will be injured only by requirements with higher priority
- We'll use *sets* instead of strings to construct A and B

The Finite Injury Priority method

The query function

- To keep track of injuries, we'll use a **query function** $\omega^A_{i,s}(x)$ that returns the largest index queried by $\Phi^A_{i,s}(x)$.

$$\omega^A_{i,s}(x) = \begin{cases} \max\{z \in \mathbb{N} \mid z \text{ gets queried in } \Phi^A_{i,s}(x)\} & \text{if } \Phi^A_{i,s}(x) \downarrow \\ -1 & \text{otherwise} \end{cases}$$

where $A \subseteq \mathbb{N}$ and $i, s, x \in \mathbb{N}$.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

Theorem

[Fri57; Muc56] There are two r.e. sets A and B that are incomparable

- Requirements:

$$R_{2j} := (\psi_j^B \neq A) \qquad R_{2j+1} := (\psi_j^A \neq B)$$

- We say that an index x is a **witness** for the requirement R_{2i} at step s if $\Phi_{i,s}^{B_s}(x) \downarrow$ and $\psi_{i,s}^{B_s}(x) \neq A_s(x)$ (symmetric definition for R_{2i+1})

The Finite Injury Priority method

The Friedberg-Muchnik theorem

Theorem

[Fri57; Muc56] There are two r.e. sets A and B that are incomparable

- Requirements:

$$R_{2j} := (\psi_j^B \neq A) \qquad R_{2j+1} := (\psi_j^A \neq B)$$

- We say that an index x is a **witness** for the requirement R_{2i} at step s if $\Phi_{i,s}^{B_s}(x) \downarrow$ and $\psi_{i,s}^{B_s}(x) \neq A_s(x)$ (symmetric definition for R_{2i+1})

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- On each step $s \in \mathbb{N}$, for each $j \in \mathbb{N}$ we compute:
  - A witness $w_{j,s}$ for R_j at step s
  - A restriction index $r_{j,s}$ to fix the priority of the requirements
- **Invariants**:
  - $w_{j,s} = r_{j,s} = -1$ when no witness is known for R_j at step s
  - Indices that come before $r_{j,s}$ don't break any requirement.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- On each step $s \in \mathbb{N}$, for each $j \in \mathbb{N}$ we compute:
  - A witness $w_{j,s}$ for R_j at step s
  - A restriction index $r_{j,s}$ to fix the priority of the requirements
- **Invariants**:
  - $w_{j,s} = r_{j,s} = -1$ when no witness is known for R_j at step s
  - Indices that come before $r_{j,s}$ don't break any requirement.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- On each step $s \in \mathbb{N}$, for each $j \in \mathbb{N}$ we compute:
  - A witness $w_{j,s}$ for R_j at step s
  - A restriction index $r_{j,s}$ to fix the priority of the requirements
- **Invariants:**
  - $w_{j,s} = r_{j,s} = -1$ when no witness is known for R_j at step s
  - Indices that come before $r_{j,s}$ don't break any requirement.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- On each step $s \in \mathbb{N}$, for each $j \in \mathbb{N}$ we compute:
  - A witness $w_{j,s}$ for R_j at step s
  - A restriction index $r_{j,s}$ to fix the priority of the requirements
- **Invariants**:
  - $w_{j,s} = r_{j,s} = -1$ when no witness is known for R_j at step s
  - Indices that come before $r_{j,s}$ don't break any requirement.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- On each step $s \in \mathbb{N}$, for each $j \in \mathbb{N}$ we compute:
  - A witness $w_{j,s}$ for R_j at step s
  - A restriction index $r_{j,s}$ to fix the priority of the requirements
- **Invariants**:
  - $w_{j,s} = r_{j,s} = -1$ when no witness is known for R_j at step s
  - Indices that come before $r_{j,s}$ don't break any requirement.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- On each step $s \in \mathbb{N}$, for each $j \in \mathbb{N}$ we compute:
  - A witness $w_{j,s}$ for R_j at step s
  - A restriction index $r_{j,s}$ to fix the priority of the requirements
- **Invariants**:
  - $w_{j,s} = r_{j,s} = -1$ when no witness is known for R_j at step s
  - Indices that come before $r_{j,s}$ don't break any requirement.

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Let $A_0 = B_0 = \varnothing$ and let $w_{j,0} = r_{j,0} = -1$ for each $j \in \mathbb{N}$.
- Consider a generic step $s > 0$
- For each $j \leq s$, if $w_{j,s-1} \neq -1$ we propagate the previous values because R_{2i} already has a witness.
  - $A_s = A_{s-1}$ and $B_s = B_{s-1}$
  - $w_{j,s} = w_{j,s-1}$ and $r_{j,s} = r_{j,s-1}$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Let $A_0 = B_0 = \varnothing$ and let $w_{j,0} = r_{j,0} = -1$ for each $j \in \mathbb{N}$.
- Consider a generic step $s > 0$
- For each $j \leq s$, if $w_{j,s-1} \neq -1$ we propagate the previous values because R_{2i} already has a witness.
  - $A_s = A_{s-1}$ and $B_s = B_{s-1}$
  - $w_{j,s} = w_{j,s-1}$ and $r_{j,s} = r_{j,s-1}$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Let $A_0 = B_0 = \varnothing$ and let $w_{j,0} = r_{j,0} = -1$ for each $j \in \mathbb{N}$.
- Consider a generic step $s > 0$
- For each $j \leq s$, if $w_{j,s-1} \neq -1$ we propagate the previous values because R_{2i} already has a witness.
  - $A_s = A_{s-1}$ and $B_s = B_{s-1}$
  - $w_{j,s} = w_{j,s-1}$ and $r_{j,s} = r_{j,s-1}$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Let $A_0 = B_0 = \varnothing$ and let $w_{j,0} = r_{j,0} = -1$ for each $j \in \mathbb{N}$.
- Consider a generic step $s > 0$
- For each $j \leq s$, if $w_{j,s-1} \neq -1$ we propagate the previous values because R_{2i} already has a witness.
  - $A_s = A_{s-1}$ and $B_s = B_{s-1}$
  - $w_{j,s} = w_{j,s-1}$ and $r_{j,s} = r_{j,s-1}$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Let $A_0 = B_0 = \varnothing$ and let $w_{j,0} = r_{j,0} = -1$ for each $j \in \mathbb{N}$.
- Consider a generic step $s > 0$
- For each $j \leq s$, if $w_{j,s-1} \neq -1$ we propagate the previous values because R_{2i} already has a witness.
  - $A_s = A_{s-1}$ and $B_s = B_{s-1}$
  - $w_{j,s} = w_{j,s-1}$ and $r_{j,s} = r_{j,s-1}$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Suppose now that $w_{2h,s-1} = -1$.
- Let x be the minimum index such that $x \notin A_{s-1}$ and such that for all $k < 2h$ it holds that $x > r_{k,s}$.
- If $\Phi_{h,s-1}^{B_{s-1}}(x) \uparrow$, we preserve that $w_{2h,s} = r_{2h,s} = -1$ and propagate $A_s = A_{s-1}$, $B_s = B_{s-1}$. This trivially satisfies the requirement R_{2h}.
- Otherwise, if $\psi_{h,s-1}^{B_{s-1}}(x) = 0$, we set:

$$w_{2h,s} = x \qquad r_{2h,s} = \max(x, \omega_{h,s-1}^{B_{s-1}}(x))$$

$$A_s = A_{s-1} \cup \{x\} \qquad B_s = B_{s-1}$$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Suppose now that $w_{2h,s-1} = -1$.
- Let x be the minimum index such that $x \notin A_{s-1}$ and such that for all $k < 2h$ it holds that $x > r_{k,s}$.
- If $\Phi_{h,s-1}^{B_{s-1}}(x) \uparrow$, we preserve that $w_{2h,s} = r_{2h,s} = -1$ and propagate $A_s = A_{s-1}$, $B_s = B_{s-1}$. This trivially satisfies the requirement R_{2h}.
- Otherwise, if $\psi_{h,s-1}^{B_{s-1}}(x) = 0$, we set:

$$w_{2h,s} = x \qquad r_{2h,s} = \max(x, \omega_{h,s-1}^{B_{s-1}}(x))$$

$$A_s = A_{s-1} \cup \{x\} \qquad B_s = B_{s-1}$$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Suppose now that $w_{2h,s-1} = -1$.
- Let x be the minimum index such that $x \notin A_{s-1}$ and such that for all $k < 2h$ it holds that $x > r_{k,s}$.
- If $\Phi_{h,s-1}^{B_{s-1}}(x) \uparrow$, we preserve that $w_{2h,s} = r_{2h,s} = -1$ and propagate $A_s = A_{s-1}$, $B_s = B_{s-1}$. This trivially satisfies the requirement R_{2h}.
- Otherwise, if $\psi_{h,s-1}^{B_{s-1}}(x) = 0$, we set:

$$w_{2h,s} = x \qquad r_{2h,s} = \max(x, \omega_{h,s-1}^{B_{s-1}}(x))$$

$$A_s = A_{s-1} \cup \{x\} \qquad B_s = B_{s-1}$$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- Suppose now that $w_{2h,s-1} = -1$.
- Let x be the minimum index such that $x \notin A_{s-1}$ and such that for all $k < 2h$ it holds that $x > r_{k,s}$.
- If $\Phi_{h,s-1}^{B_{s-1}}(x) \uparrow$, we preserve that $w_{2h,s} = r_{2h,s} = -1$ and propagate $A_s = A_{s-1}$, $B_s = B_{s-1}$. This trivially satisfies the requirement R_{2h}.
- Otherwise, if $\psi_{h,s-1}^{B_{s-1}}(x) = 0$, we set:

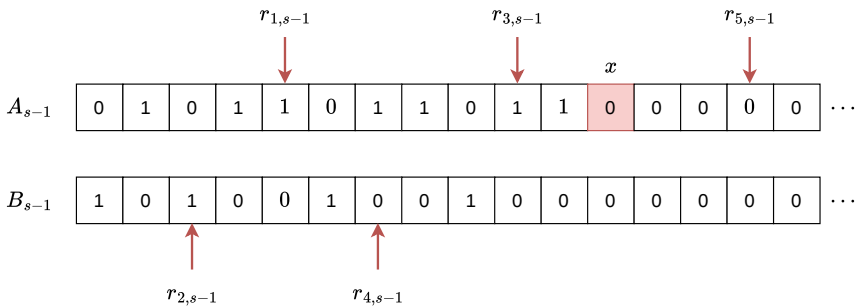
$$w_{2h,s} = x \qquad r_{2h,s} = \max(x, \omega_{h,s-1}^{B_{s-1}}(x))$$

$$A_s = A_{s-1} \cup \{x\} \qquad B_s = B_{s-1}$$

The Finite Injury Priority method

The Friedberg-Muchnik theorem

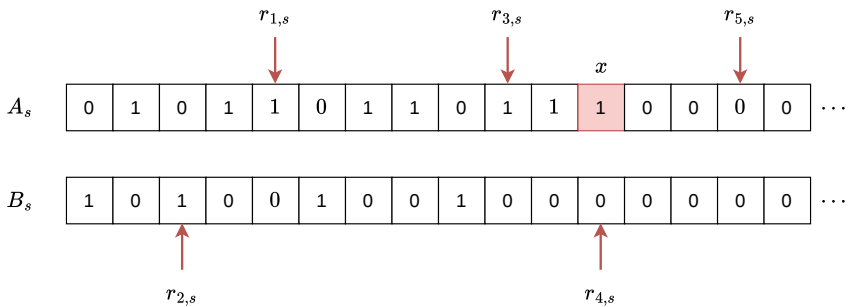
What happens when $w_{4,s} = -1$?



The Finite Injury Priority method

The Friedberg-Muchnik theorem

What happens when $w_{4,s} = -1$?



The Finite Injury Priority method

The Friedberg-Muchnik theorem

Claim: For each $j \in \mathbb{N}$ it holds that:

1. R_j is injured at most $2^j - 1$ times
2. There is a step s_0 such that for all $s \geq s_0$ and for all $k < j$ no new index is added to A_s

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- $A = \bigcup_{s=0}^{+\infty} A_s$ and $B = \bigcup_{s=0}^{+\infty} B_s$
- The claim implies that every requirement will be, eventually, satisfied by the construction.
- Moreover, the use of the restriction values $r_{j,s}$ allows us to recursively enumerate the sets A and B by restricting our interest to the indexes between 0 and $r_{j,s}$ for each $j \in \mathbb{N}$
  - This implies that A and B are both r.e.!

□

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- $A = \bigcup_{s=0}^{+\infty} A_s$ and $B = \bigcup_{s=0}^{+\infty} B_s$
- The claim implies that every requirement will be, eventually, satisfied by the construction.
- Moreover, the use of the restriction values $r_{j,s}$ allows us to recursively enumerate the sets A and B by restricting our interest to the indexes between 0 and $r_{j,s}$ for each $j \in \mathbb{N}$
  - This implies that A and B are both r.e.!

□

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- $A = \bigcup_{s=0}^{+\infty} A_s$ and $B = \bigcup_{s=0}^{+\infty} B_s$
- The claim implies that every requirement will be, eventually, satisfied by the construction.
- Moreover, the use of the restriction values $r_{j,s}$ allows us to recursively enumerate the sets A and B by restricting our interest to the indexes between 0 and $r_{j,s}$ for each $j \in \mathbb{N}$
  - This implies that A and B are both r.e.!

□

The Finite Injury Priority method

The Friedberg-Muchnik theorem

- $A = \bigcup_{s=0}^{+\infty} A_s$ and $B = \bigcup_{s=0}^{+\infty} B_s$
- The claim implies that every requirement will be, eventually, satisfied by the construction.
- Moreover, the use of the restriction values $r_{j,s}$ allows us to recursively enumerate the sets A and B by restricting our interest to the indexes between 0 and $r_{j,s}$ for each $j \in \mathbb{N}$
  - This implies that A and B are both r.e.!

□

The Finite Injury Priority method

Other interesting results

- **Infinite Injury Priority Argument**: Sacks [Sac64] proved that the above construction can be extended to a countably infinite argument
- **Density of r.e. sets**: For each pair of r.e. sets A, B there is another r.e. set C such that $A <_T C <_T B$
- Simpson [Sim77] proved that the first-order theory of $\mathcal{D}$ over the language $(\preceq, =)$ is many-one equivalent to the theory of true second-order arithmetic.

The Finite Injury Priority method

Other interesting results

- **Infinite Injury Priority Argument**: Sacks [Sac64] proved that the above construction can be extended to a countably infinite argument
- **Density of r.e. sets**: For each pair of r.e. sets A, B there is another r.e. set C such that $A <_T C <_T B$
- Simpson [Sim77] proved that the first-order theory of $\mathcal{D}$ over the language $(\preceq, =)$ is many-one equivalent to the theory of true second-order arithmetic.

The Finite Injury Priority method

Other interesting results

- **Infinite Injury Priority Argument**: Sacks [Sac64] proved that the above construction can be extended to a countably infinite argument
- **Density of r.e. sets**: For each pair of r.e. sets A, B there is another r.e. set C such that $A <_T C <_T B$
- Simpson [Sim77] proved that the first-order theory of $\mathcal{D}$ over the language $(\preceq, =)$ is many-one equivalent to the theory of true second-order arithmetic.

Thank you for listening!
Any questions?

Bibliography

[Fri57] Richard M. Friedberg. “Two recursively enumerable sets of incomparable degrees of unsolvability (solution of Post’s problem, 1944)”. In: (1957).

[KP54] Stephen C. Kleene and Emil L. Post. “The upper semi-lattice of degrees of recursive unsolvability”. In: (1954).

[Muc56] Albert A. Muchnik. “On the unsolvability of the problem of reducibility in the theory of algorithms”. In: (1956).

[Pos44] Emil L. Post. “Recursively enumerable sets of positive integers and their decision problems”. In: (1944).

[Sac64] Gerald E. Sacks. “The Recursively Enumerable Degrees are Dense”. In: (1964).

[Sim77] Stephen G. Simpson. “First-Order Theory of the Degrees of Recursive Unsolvability”. In: (1977).

[Tur36] Alan M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: (1936).